

UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Triennale in Informatica

Progettazione Architettuale
Quantum Specific Code Smells
Optimization Pipeline

Manuel Giordano
Matricola: 0512116049

Anno Accademico 2025/2026

Chapter 1

Progettazione Architettuale del Sistema

1.1 Introduzione e panoramica dell'architettura

Il presente progetto di tesi si pone l'obiettivo di definire e implementare un'architettura software in grado di identificare, refattorizzare e validare anomalie note come *Quantum Code Smells* all'interno di codici quantistici scritti utilizzando il framework IBM Qiskit. Il sistema è stato progettato seguendo un approccio modulare basato su *Component-Based Development* per far fronte alla natura sperimentale e in rapida evoluzione del dominio quantistico, garantendo la flessibilità necessaria per sostituire, aggiornare o estendere i singoli moduli senza impattare l'intero ecosistema. L'architettura del sistema è logicamente suddivisa in tre macro-moduli indipendenti:

1. **Quantum Circuit Extraction and Profiling (QCEP)**: Un modulo di ingestione dei dati dedicato all'estrazione dei circuiti quantistici e al calcolo delle metriche *baseline* tramite Qiskit. Produce un file di formato JSONL.
2. **Quantum Specific Code Smells Optimization Pipeline (QSCSOP)**: Il core computazionale del sistema. Il modulo orchestra tre componenti interni: un adattatore di ingresso, un motore multi-agente e un componente di persistenza tramite interfacce astratte. Il motore multi-agente, implementato con il framework CrewAI, si occupa di rilevare le anomalie strutturali presenti nei circuiti, produrne una versione ottimizzata e validarne la correttezza tramite Qiskit. I risultati del processo vengono serializzati in un file di formato JSONL.
3. **Analytics**: Un componente di post-elaborazione indipendente. Si occupa di ricevere in input dati serializzati dalla fase precedente per calcolare le metriche di valutazione empirica tramite Pandas. Il processo si conclude con la generazione automatizzata dei grafici e la loro esportazione in formato PDF.

Al fine di garantire un elevato grado di isolamento tra le fasi e prevenire la perdita di dati durante elaborazioni computazionalmente onerose, i moduli non comunicano in modo sincrono, ma scambiano informazioni attraverso la persistenza di file intermedi. A livello implementativo, l'orchestrazione del sistema rispetta rigorosamente il principio dell'Inversione delle Dipendenze (*Dependency Inversion*): i moduli core dialogano esclusivamente tramite interfacce astratte. Sebbene il formato di interscambio predefinito sia il JSON Lines (.jsonl), l'architettura supporta nativamente qualsiasi altro formato, essendo sufficiente iniettare una nuova implementazione concreta dell'interfaccia di I/O. Questa solidità architettuale, unita all'adozione del pattern *Generator* e a scritture incrementalmente su disco (in modalità *append*), garantisce un'occupazione di memoria altamente ottimizzata, mantenendo una complessità spaziale pari a $O(1)$ rispetto alla dimensione del dataset durante l'intero ciclo.

1.2 Analisi del dominio dei dati

La validità di un'architettura orientata all'analisi e al refactoring del codice dipende in primo luogo dalla solidità e dalla natura dei dati forniti in ingresso. Per questo motivo, la definizione del dominio applicativo ha richiesto l'identificazione di sorgenti e formati in grado di supportare in modo affidabile l'intera pipeline di elaborazione.

1.2.1 Criteri di ammissibilità e dataset selezionati

La ricerca dei dataset si è concentrata su fonti in grado di offrire codice quantistico, eseguibile, *human-written* e scritto esclusivamente utilizzando il framework IBM Qiskit. In quest'ottica la scelta è ricaduta su due dataset complementari: il benchmark Bugs4Q e il dataset analizzato nello studio TheSmellyEight. Il primo fornisce un buon numero di circuiti quantistici; per ragioni puramente architetturali, sono state estratte in fase di *parsing* esclusivamente le revisioni contrassegnate come risolte (*fixed*). Questa limitazione ai soli codici funzionanti rappresenta una scelta ingegneristica precisa, in quanto processare script intrinsecamente difettosi non solo rischierebbe di causare fallimenti a catena durante la fase di isolamento del circuito tramite la funzione `exec()`, ma porterebbe al calcolo di metriche basate su logiche fallate alla radice, rendendo del tutto vano il successivo tentativo di ottimizzazione da parte del sistema multi-agente. La seconda sorgente, al contrario, offre un terreno di prova già mirato e specifico per il modulo QSCSOP, racchiudendo casi di studio in cui sono documentate anomalie strutturali latenti su cui gli agenti possono intervenire direttamente effettuandone il refactoring.

1.2.2 Trasformazione e modellazione del dato

Lo scopo del primo modulo dell'architettura è proprio quello di digerire i dati appena descritti per costruire una base di partenza quantificabile. In fase di ingestione il parser del relativo dataset manipola un record estremamente minimale, il cui unico scopo è trasportare il codice sorgente grezzo accompagnato dal proprio identificativo e dalla provenienza del dataset.

```
{
  "circuitId": "...",
  "datasetSource": "...",
  "sourceCode": "..."
}
```

Listing 1.1: Esempio del record grezzo in fase di ingestione

Attraversando la pipeline del modulo QCEP, questa semplice stringa di testo viene tradotta in un reale oggetto circuito e arricchita con le valutazioni di base fornite da `Qiskit`, per essere infine serializzata su disco. Al termine di questa operazione il dato assume la forma di un profilo strutturato e gerarchico salvato in formato JSON Lines.

Per ragioni di chiarezza espositiva, il Listing 1.2 illustra la struttura logica di un singolo record espanso; nel file di output reale, al fine di garantire i requisiti di memoria precedentemente descritti, ciascuno di questi oggetti viene minificato e serializzato in *append* su una singola riga nel formato JSON Lines.

```

{
  "circuitId": "bug_1",
  "datasetSource": "Bugs4Q",
  "baseline": {
    "sourceCode": "from qiskit import *...",
    "logicalQubits": 3,
    "abstractMetrics": {
      "gateCount": 15,
      "depth": 8
    },
    "physicalMetrics": {
      "gateCount": 42,
      "depth": 25
    }
  }
}

```

Listing 1.2: Rappresentazione gerarchica della struttura di un singolo record JSON Lines prodotto dal modulo QCEP

Questo nuovo “contratto” racchiude al suo interno il codice originale, la configurazione dei qubit logici e le metriche astratte e fisiche fondamentali come *gate count* e *depth*, stabilendo la *baseline* necessaria per misurare in modo oggettivo il successo delle successive ottimizzazioni.

1.3 Il modulo QCEP: Quantum Circuit Extraction and Profiling

Il modulo QCEP rappresenta il motore di ingestione e pre-elaborazione dell’intera architettura. Progettato come una classica pipeline offline di tipo Extract, Transform, Load (ETL), questo ecosistema è interamente governato da un controllore centrale, denominato **QCEPMain**, il quale orchestra il ciclo di vita del dato garantendo che il consumo di memoria rimanga costante indipendentemente dalla mole dei repository analizzati.

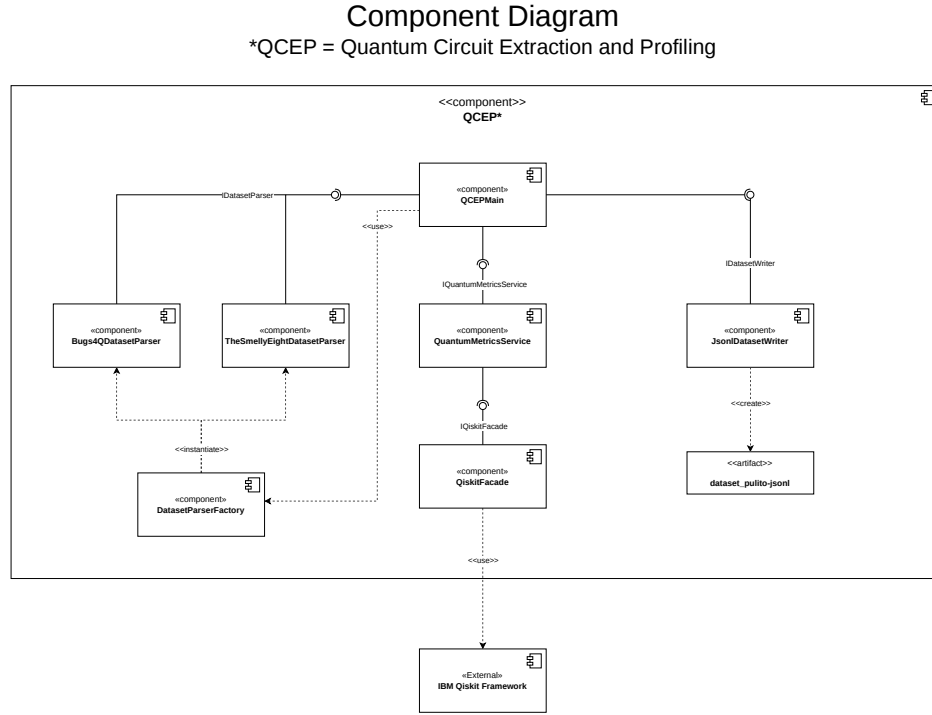


Figure 1.1: Component diagram del modulo QCEP

1.3.1 Orchestrazione e Pattern Factory (Fase di Estrazione)

L'architettura del modulo è organizzata attorno a una netta separazione delle responsabilità. Il componente principale, `QCEPMain`, ha il compito di inizializzare le dipendenze e coordinare il flusso dei dati, senza occuparsi direttamente dei calcoli di dominio. Per garantire estensibilità verso repository futuri, il sistema adotta il pattern Factory: tramite una `DatasetParserFactory`, viene istanziato dinamicamente il parser appropriato (ad esempio `Bugs4QDatasetParser`) astruendo la logica di navigazione delle directory dal resto del sistema.

Il processo di estrazione si basa sui generatori nativi di Python, che trasformano la lettura dei file in un flusso incrementale: il parser elabora un singolo script alla volta, lo incapsula in un record grezzo e cede il controllo al livello superiore, sospendendo la propria esecuzione fino alla richiesta successiva. Questo approccio evita il caricamento in blocco dell'intero dataset in memoria, mantenendo l'occupazione della RAM costante per tutta la durata dell'esecuzione.

1.3.2 Validazione e Isolamento Quantistico (Fase di Trasformazione)

La fase di trasformazione è gestita dal componente `QuantumMetricsService`, che riceve i record grezzi dal generatore e ne coordina l'analisi. Prima di procedere, ogni script viene sottoposto a una validazione sintattica tramite la libreria nativa `ast` (Abstract Syntax Tree), che consente di

individuare errori grammaticali o di indentazione senza eseguire il codice: qualora la validazione fallisca, il record viene scartato dal flusso senza interrompere l'elaborazione degli elementi successivi.

Gli script validi vengono passati alla **QiskitFacade**, che incapsula tutte le dipendenze legate al framework IBM Qiskit, isolando la complessità della libreria dai livelli superiori del sistema ed evitando il problema della *Leaky Abstraction*. Al suo interno, il codice sorgente viene eseguito in una sandbox controllata tramite **exec()**, al fine di istanziare in memoria l'oggetto **QuantumCircuit** corrispondente. Dal circuito estratto vengono quindi ricavate le metriche astratte (numero di qubit logici, gate count e depth originaria) dopodiché si procede alla transpilazione su un simulatore hardware (*Fake Backend*), che permette di misurare le metriche fisiche tenendo conto delle operazioni di routing e delle porte SWAP introdotte per adattare il circuito a una topologia concreta.

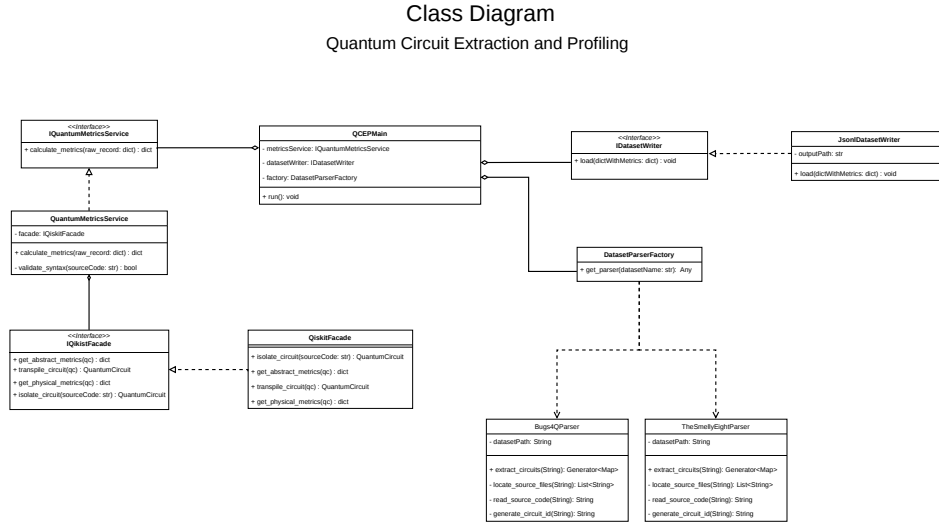


Figure 1.2: Class diagram del modulo QCEP

1.3.3 Persistenza dei Dati (Fase di Load)

La fase di caricamento è affidata al componente **JsonDatasetWriter**, implementazione concreta dell'interfaccia astratta di scrittura del sistema. Una volta ricevute le metriche validate, il componente si occupa del mapping dei dati nella struttura gerarchica descritta in precedenza.

Per mantenere una complessità spaziale pari a $\mathcal{O}(1)$, i risultati non vengono accumulati in una struttura dati globale: ogni record completato viene immediatamente serializzato e scritto su disco in modalità *append*. In questo modo la memoria viene liberata progressivamente, permettendo al sistema di processare dataset di dimensioni arbitrarie senza rischi di esaurimento della memoria.

1.4 Il modulo QSCSOP: Quantum Specific Code Smells Optimization Pipeline

Il modulo QSCSOP costituisce il nucleo computazionale dell'intera architettura. Il suo compito è ricevere in ingresso i circuiti quantistici profilati dal modulo QCEP, sottoporli a un ciclo iterativo di rilevamento e refattorizzazione dei *Quantum Code Smells*, validarne la correttezza e serializzare i risultati in un file di output strutturato, pronto per essere consumato dal modulo Analytics.

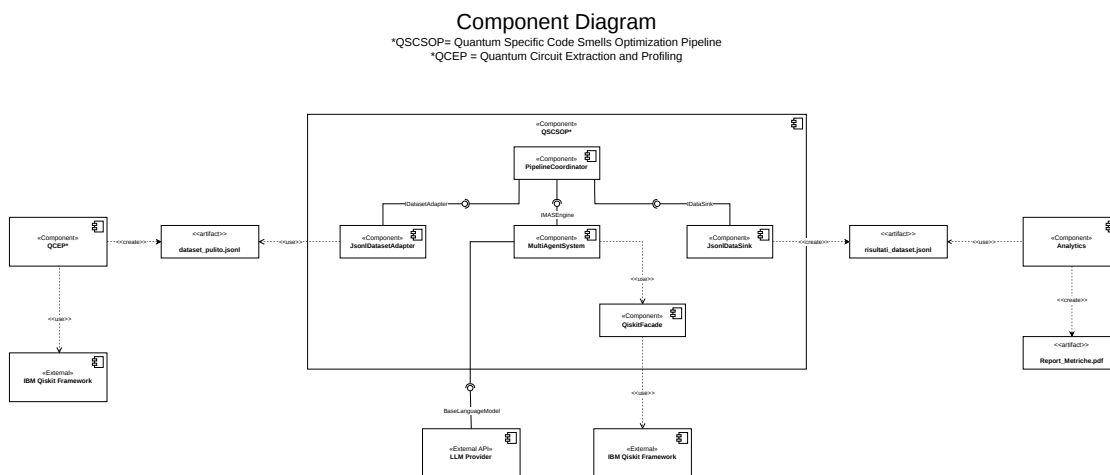


Figure 1.3: Component Diagram del modulo QSCSOP

1.4.1 Struttura e Pattern Architeturali

L'architettura del modulo è progettata attorno al principio dell'Inversione delle Dipendenze: il **PipelineCoordinator**, componente centrale che orchestra l'intero flusso, non dipende da alcuna implementazione concreta, ma dialoga esclusivamente con tre interfacce astratte: **IDatasetAdapter**, **IMASEngine** e **IDataSink**. A ciascuna di esse corrisponde un'implementazione concreta predefinita, rispettivamente **JsonDatasetAdapter**, **MASEngine** e **JsonDataSink** che può essere sostituita senza modificare la logica di orchestrazione. Questo disaccoppiamento rende il modulo aperto all'estensione: è sufficiente iniettare una nuova implementazione per supportare, ad esempio, un formato di input o output diverso dal JSON Lines, o integrare un motore agentivo alternativo.

1.4.2 L'oggetto QuantumProgramEntity e il Ciclo di Vita del Dato

L'unità di elaborazione fondamentale del modulo è la **QuantumProgramEntity**, l'oggetto che accompagna e accumula le informazioni relative a un singolo circuito quantistico per l'intera durata del suo ciclo di elaborazione all'interno di QSCSOP.

L'entità viene istanziata dal **JsonDatasetAdapter**, che legge i record prodotti dal modulo QCEP e li deserializza nella struttura oggetto corrispondente. Al momento della creazione, la **QuantumProgramEntity** contiene il codice sorgente originale, il numero di qubit logici e le metriche fisiche e astratte di *baseline* calcolate da QCEP. Questa rappresentazione strutturata costituisce il

contratto che il modulo QSCSOP mantiene internamente tra i suoi componenti: il **PipelineCoordinator** la estrae dall'adattatore, la cede al **MASEngine** per l'elaborazione, la riceve arricchita con i risultati dell'ottimizzazione e infine la consegna al **JsonlDataSink** per la serializzazione, indipendentemente dall'esito del processo.

Class Diagram
Quantum Specific Code Smell Optimization Pipeline

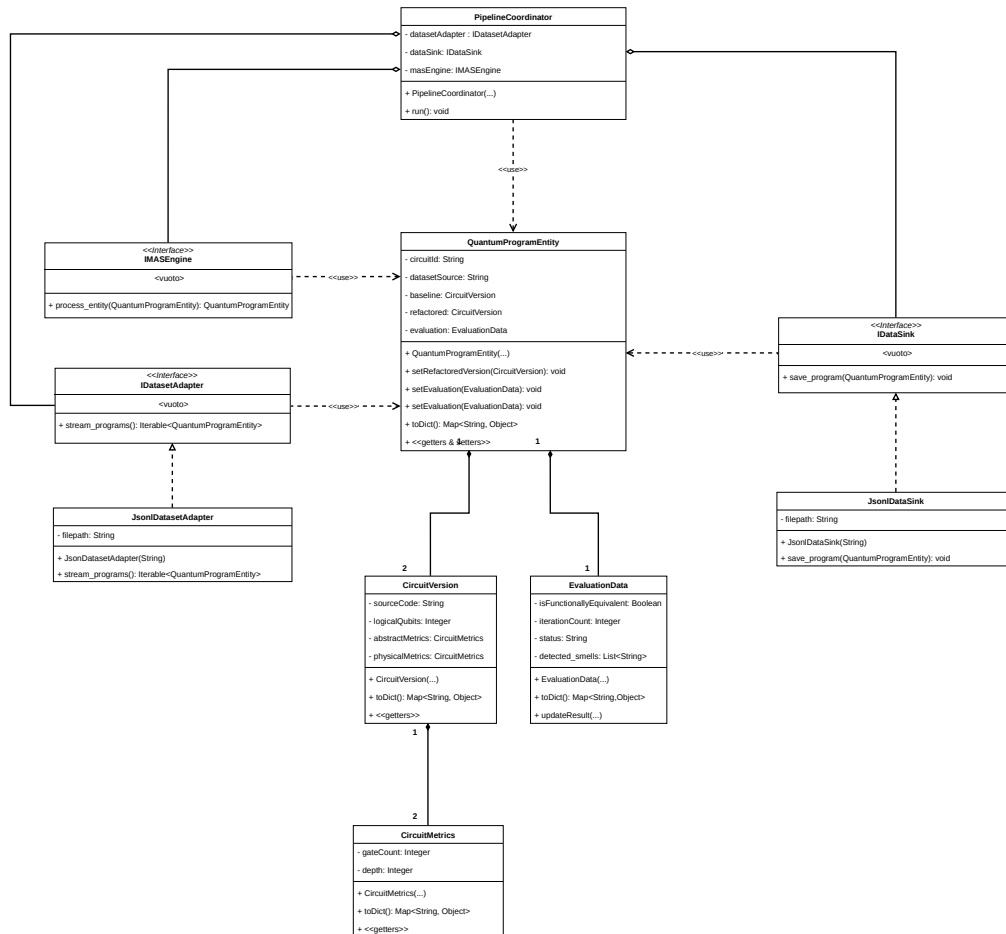


Figure 1.4: Class Diagram del modulo QSCSOP

1.4.3 Il Sottosistema MAS: Multi-Agent System

Il **MASEngine** è il componente a cui il **PipelineCoordinator** delega l'intera fase computazionale. Riceve in ingresso la **QuantumProgramEntity** e ne restituisce una versione aggiornata con i risultati dell'ottimizzazione. Internamente, il motore coordina tre agenti specializzati e un servizio di validazione deterministica, realizzando un ciclo iterativo di analisi, refattorizzazione e verifica.

L'architettura interna del MAS adotta il meccanismo di *Dependency Injection* per gli agenti: **DetectorAgent**, **RefactorerAgent** e **ReviewerAgent** dipendono dal tipo astratto **BaseLanguageModel**, fornito dal framework LangChain su cui è basato CrewAI. Questo disaccoppiamento permette di variare il modello linguistico sottostante senza modificare la logica interna degli agenti. Il sistema è implementato tramite il framework CrewAI, che consente di modellare le responsabilità operative assegnando ruoli distinti e specializzati a ciascun agente.

Il **MASEngine** agisce inoltre come *Mediator*: coordina il flusso tra gli agenti, interpreta i risultati della validazione e decide le azioni conseguenti, senza mai assumere responsabilità di calcolo.

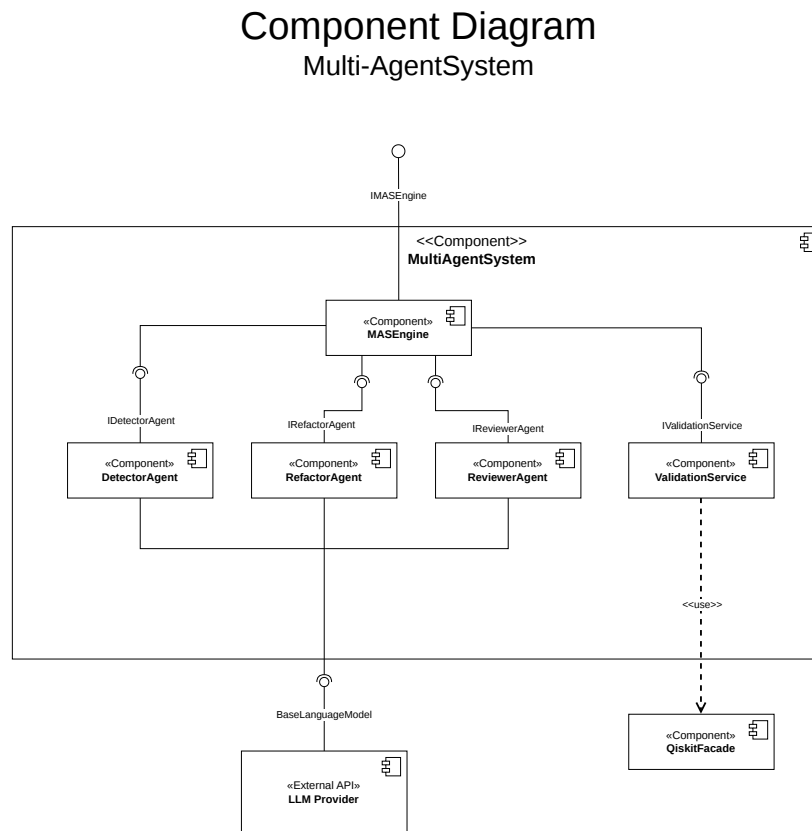


Figure 1.5: Component Diagram del sottosistema Multi-Agent System

Gli Agenti e le loro Responsabilità

Il modulo è composto da tre agenti specializzati, ciascuno con un ruolo ben definito e non intercambiabile.

Il **DetectorAgent** ha il compito esclusivo di analizzare staticamente il codice sorgente del circuito e identificare la presenza di *Quantum Code Smells*. Il suo output è un oggetto **SmellReportDTO**, che trasporta un flag booleano indicante l'esito dell'analisi e, in caso affermativo, una descrizione discorsiva delle anomalie rilevate. La scelta di incapsulare l'esito in un oggetto strutturato, anziché restituire una semplice stringa, risponde all'esigenza di garantire che le informazioni di contesto rimangano disponibili e tipizzate per tutte le fasi successive del ciclo. Qualora il flag risulti negativo, il **MASEngine** marca immediatamente la **QuantumProgramEntity** come **SMELL_FREE** e la restituisce al **PipelineCoordinator**, che ne delega la serializzazione al **JsonlDataSink** senza avviare alcun ciclo di ottimizzazione.

Il **RefactorerAgent** riceve in ingresso il codice sorgente originale e il report del **DetectorAgent** e ha il compito di produrre una versione ottimizzata del circuito che elimini le anomalie segnalate. A partire dalla seconda iterazione, il suo contesto di input viene arricchito con il feedback testuale generato dal **ReviewerAgent** in merito al tentativo precedente, realizzando un ciclo di apprendimento contestuale che guida progressivamente la generazione verso soluzioni valide.

Il **ReviewerAgent** non interviene in ogni ciclo di esecuzione, ma rappresenta un componente attivato selettivamente dal **MASEngine** esclusivamente nelle iterazioni in cui la validazione del codice prodotto dal **RefactorerAgent** non ha avuto esito positivo. La distinzione di ruolo rispetto agli altri agenti è netta: mentre il **DetectorAgent** e il **RefactorerAgent** operano in fasi distinte e sequenziali della pipeline, il **ReviewerAgent** si colloca in una posizione reattiva, intervenendo solo quando il ciclo di ottimizzazione ha prodotto un risultato non conforme ai criteri stabiliti.

Il **ValidationService**, componente deterministico del sistema, restituisce al termine di ogni validazione un **ValidationResultDTO** contenente due campi informativi: un valore booleano che certifica l'esito del processo (**is_valid**) e una stringa descrittiva che riporta, a seconda del caso, il traceback dell'eccezione Python sollevata durante l'esecuzione oppure la motivazione del fallimento del controllo di equivalenza funzionale. Trasmettere queste informazioni direttamente al **RefactorerAgent** imporrebbe all'agente di interpretare autonomamente un errore di sistema a basso livello, senza alcun riferimento all'intenzione progettuale che aveva guidato il tentativo di refactoring precedente. In assenza di tale contestualizzazione, il rischio è che l'agente reagisca all'errore in modo non mirato, producendo una correzione disallineata rispetto agli obiettivi originari di qualità del circuito.

Il **ReviewerAgent** è progettato per colmare questo gap; ricevendo in input lo **SmellReportDTO** originale (generato dal **DetectorAgent** nella fase iniziale di rilevazione dei Quantum Code Smells) e il **ValidationResultDTO** prodotto dal **ValidationService**, l'agente non si limita a ritrasmettere l'errore grezzo, bensì lo contestualizza rispetto all'intento del refactoring fallito. L'output prodotto dal **ReviewerAgent** costituisce dunque una sintesi contestualizzata dell'errore rilevato, formulata in modo da rendere il feedback immediatamente utilizzabile dal **RefactorerAgent** nell'iterazione successiva, senza che quest'ultimo debba ricavarne autonomamente le implicazioni rispetto alla strategia di refactoring adottata.

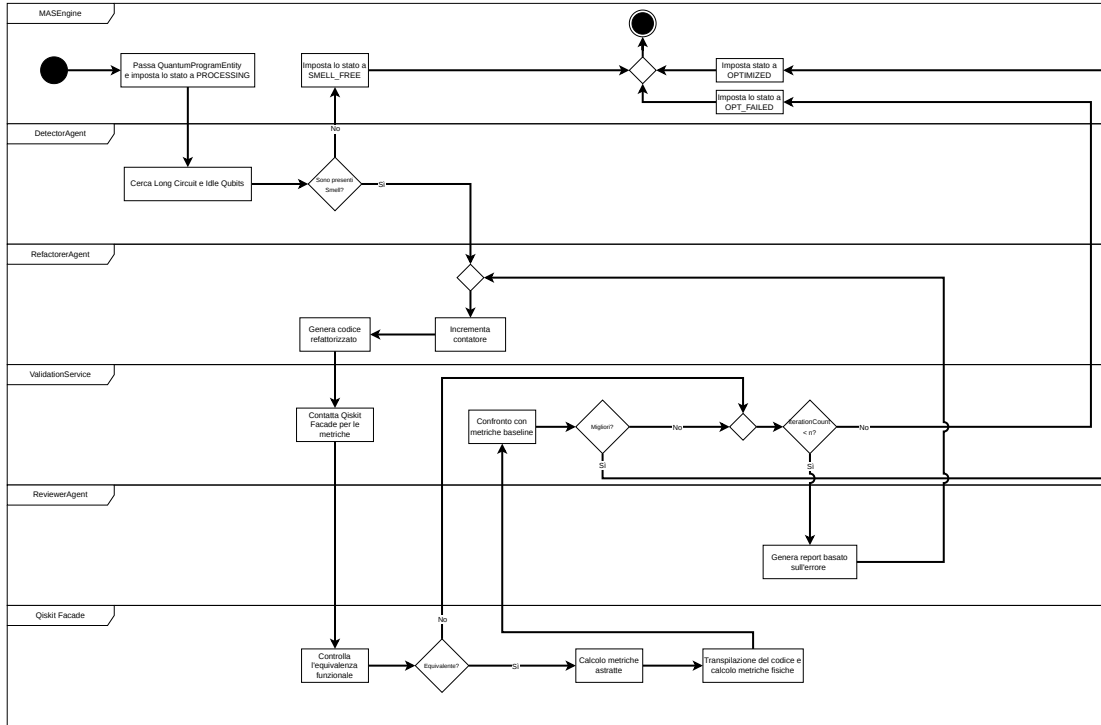


Figure 1.6: Activity Diagram del Multi-Agent System

Il ValidationService: Pipeline di Verifica Deterministica

La validazione del codice generato è affidata al **ValidationService**, che esegue una sequenza di controlli ordinata e strettamente sequenziale. La natura deterministica di questa pipeline è una scelta progettuale deliberata: affidare la verifica della correttezza a un componente privo di elementi stocastici garantisce la riproducibilità dei risultati e consente di produrre messaggi di errore precisi e strutturati, fondamentali per l'operato del **ReviewerAgent**.

Il **ValidationService** contatta la **QiskitFacade** per verificare la correttezza sintattica e semantica, prima di calcolare le metriche astratte, transpilare il circuito e calcolare le metriche fisiche. Il primo passo esegue la validazione sintattica del codice generato dal **RefactorerAgent** tramite la creazione di un ambiente di esecuzione isolato locale. All'interno di questo spazio protetto, la stringa di codice quantistico viene interpretata ed eseguita dinamicamente sfruttando i meccanismi nativi di Python. Questo approccio a *sandbox* permette di intercettare in modo sicuro qualsiasi anomalia, da semplici errori di indentazione a chiamate verso *gate* inesistenti o violazioni delle API di Qiskit. In caso di fallimento, l'eccezione viene catturata per estrarre il *traceback* grezzo dell'errore; superato questo controllo preliminare si passa al controllo dell'equivalenza funzionale confrontando i vettori di stato (*Statevector*) prodotti dal circuito originale e da quello refactorizzato. Con esito positivo si passa al calcolo delle metriche astratte del nuovo circuito, dopodiché ne viene eseguita la transpilazione su un simulatore hardware (*Fake Backend*) per ricavarne le metriche fisiche *depth* e *gate count*. L'esito dell'intera pipeline viene infine ritornato al **ValidationService** e successivamente incapsulato in

un `ValidationResultDTO`, che trasporta il flag di validità, l'eventuale messaggio di errore e le nuove metriche, fornendo al `MASengine` tutte le informazioni necessarie per determinare il passo successivo.

Ciclo Iterativo e Gestione della Convergenza

L'intero processo di ottimizzazione è governato da un ciclo iterativo controllato dal `MASengine`. Ad ogni iterazione, il motore interpreta il `ValidationResultDTO` ricevuto: un esito positivo chiude il ciclo aggiornando la `QuantumProgramEntity` con il nuovo codice e le nuove metriche e marcandola come `OPTIMIZED`, mentre un esito negativo attiva il `ReviewerAgent` e rilancia il ciclo incrementando il contatore `iterationCount`. Per prevenire l'esecuzione indefinita in caso di mancata convergenza, il sistema adotta un limite superiore parametrico `maxIterations`: al suo superamento, l'entità viene marcata come `OPT_FAILED` e restituita al `PipelineCoordinator` senza ulteriori elaborazioni. Questa condizione di guardia garantisce che il modulo produca sempre un output terminale, indipendentemente dalla difficoltà del caso trattato.

1.4.4 Persistenza dell'Output

Al termine dell'elaborazione, il `PipelineCoordinator` riceve la `QuantumProgramEntity` aggiornata e ne affida la serializzazione al `JsonlDataSink`, implementazione concreta dell'interfaccia `IDataSink`. Indipendentemente dallo stato finale raggiunto dall'entità — `SMELL_FREE`, `OPTIMIZED` o `OPT_FAILED` — il record viene appeso al file di output in modalità *append* e in forma minificata, mantenendo una complessità spaziale pari a $\mathcal{O}(1)$ per tutta la durata dell'elaborazione. Il flusso riprende quindi con l'entità successiva estratta dall'`IDatasetAdapter`.

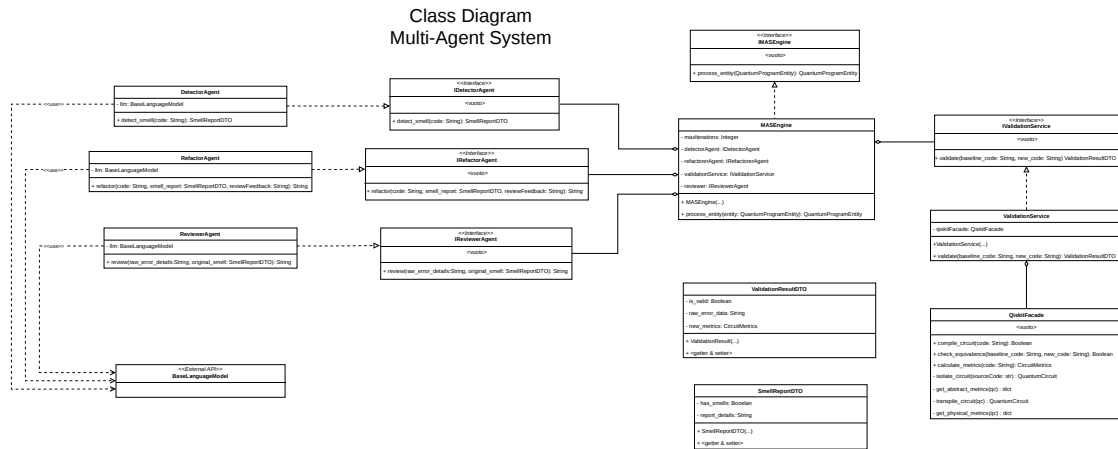


Figure 1.7: Class Diagram del sottosistema Multi-Agent System

1.5 Il modulo Analytics

Il modulo Analytics costituisce la fase terminale della pipeline complessiva e si occupa di trasformare i dati serializzati dal modulo QSCSOP in metriche aggregate e artefatti visivi pronti per l'analisi

empirica. Il modulo è orchestrato dal componente centrale **AnalyticsMain**, che coordina il flusso attraverso tre interfacce astratte **IDataLoader**, **IMetricsCalculator** e **IReportVisualizer** ciascuna con la propria implementazione concreta. Analogamente agli altri moduli dell'architettura, l'adozione sistematica di interfacce garantisce che i singoli componenti siano sostituibili indipendentemente, rispettando il principio di Inversione delle Dipendenze che governa l'intero sistema.

1.5.1 DataLoader: Ingestione e Appiattimento

Il **DataLoader** è responsabile dell'ingestione del file `risultati_dataset.jsonl` prodotto da QSCSOP. Il componente legge il file riga per riga e, sfruttando la funzione `json_normalize()` della libreria Pandas, trasforma la struttura gerarchica di ciascun record JSON in una rappresentazione bidimensionale. L'output di questa fase è un **DataFrame** in cui ogni riga corrisponde a un circuito e ogni colonna a una metrica.

Un vantaggio architetturale rilevante di questo approccio riguarda la gestione delle assenze di dati. I circuiti marcati come **SMELL_FREE** dal modulo QSCSOP non possiedono il blocco **refactored** nel loro record, in quanto il ciclo di ottimizzazione non è stato avviato. Pandas gestisce nativamente questa eterogeneità strutturale popoland automaticamente le celle mancanti con il valore **NaN** (*Not a Number*), impedendo che l'assenza di una chiave provochi errori a catena nelle fasi successive e rendendo il **DataFrame** uniforme indipendentemente dallo stato finale di ciascun circuito.

1.5.2 MetricsCalculator: Il Motore Matematico

Il **MetricsCalculator** riceve in ingresso il **DataFrame** prodotto dal **DataLoader** e si occupa del calcolo delle metriche di valutazione empirica definite nella Sezione 1.6. Le operazioni sono implementate in forma vettorializzata, ovvero attraverso filtri e aggregazioni eseguiti direttamente sulle colonne del **DataFrame** senza ricorrere a cicli iterativi espliciti, sfruttando appieno le ottimizzazioni interne di Pandas. Per garantire la significatività statistica dei risultati, il componente applica a ciascuna metrica il filtro sullo stato finale della **QuantumProgramEntity**: i miglioramenti sulle metriche fisiche vengono calcolati esclusivamente sui circuiti che hanno raggiunto lo stato **OPTIMIZED**, escludendo i casi **OPT_FAILED** e **SMELL_FREE** che introdurrebbero valori non comparabili nell'aggregazione. L'output del componente è un dizionario Python contenente i valori numerici aggregati e normalizzati, pronti per essere consumati dal **ReportVisualizer**.

I dati prodotti dal **MetricsCalculator** sono indipendenti dalla loro destinazione e possono essere utilizzati per qualsiasi scopo (stampa a terminale, generazione di tabelle testuali o trasmissione al visualizzatore) senza modificare la logica di calcolo.

1.5.3 ReportVisualizer: Il Generatore di Artefatti

Il **ReportVisualizer** riceve in ingresso sia il **DataFrame**, per i grafici distribuzionali, sia il dizionario delle metriche aggregate prodotto dal **MetricsCalculator**, per i grafici riassuntivi, e si occupa della generazione automatizzata degli artefatti visivi. Il componente utilizza le librerie Matplotlib e Seaborn, quest'ultima configurata con il contesto **paper** (modalità progettata specificamente per la produzione di figure destinate a pubblicazioni scientifiche, che impone proporzioni, dimensioni dei font e densità visiva coerenti con gli standard tipografici accademici). Tramite queste librerie vengono prodotti boxplot, grafici a barre e istogrammi per la rappresentazione delle metriche.

L'output del componente è costituito esclusivamente da file in formato **.pdf**, esportati fisicamente su disco al termine dell'elaborazione.

1.6 Metriche di Valutazione del Sistema

Al fine di mantenere la complessità del progetto entro limiti compatibili con gli obiettivi di tesi, garantendo al contempo un approccio scientifico rigoroso, la valutazione del framework si concentra in modo mirato sui due *Quantum Code Smells* più critici per i dispositivi NISQ: il *Long Circuit* e gli *Idle Qubits*. Il successo del sistema viene misurato attraverso tre categorie di metriche oggettive e complementari.

La prima categoria riguarda la correttezza e la capacità di risoluzione del sistema. Il *Tasso di Risoluzione* quantifica in quante istanze il MAS riesce a identificare e rimuovere con successo le inefficienze legate ai due smell analizzati, producendo un circuito refattorizzato con metriche fisiche migliori rispetto alla *baseline*. A questa si affianca una metrica stringente di *Equivalenza Funzionale*: ogni circuito refattorizzato viene simulato per verificare che il processo di ottimizzazione non abbia alterato lo stato quantistico o le distribuzioni di probabilità originali, garantendo la totale preservazione della semantica dell'algoritmo. Entrambe queste misurazioni sono calcolate esclusivamente sui circuiti che hanno attraversato il ciclo di ottimizzazione, escludendo le istanze marcate come `SMELL_FREE` dal `DetectorAgent`.

La seconda categoria riguarda i miglioramenti fisico-strutturali del circuito. Poiché *Long Circuit* e *Idle Qubits* impattano direttamente le probabilità di successo dell'esecuzione su hardware reale, la valutazione misura le variazioni delle metriche fisiche prima e dopo il refactoring. Per validare la neutralizzazione del *Long Circuit* vengono misurate la percentuale di riduzione della *depth* e del *gate count* fisici, ricavati tramite transpilazione su *Fake Backend*. Per confermare l'eliminazione degli *Idle Qubits* viene invece confrontato il numero di qubit logici del circuito originale con quello del circuito refattorizzato, entrambi misurati prima della transpilazione, in modo da riflettere la struttura astratta del circuito al netto delle operazioni di routing introdotte dal transpilatore.

La terza categoria riguarda le prestazioni intrinseche del framework multi-agente. Il *Tasso di Successo Globale* misura la frequenza con cui il sistema porta a termine il processo producendo codice Qiskit sintatticamente valido, compilabile e privo di allucinazioni strutturali, indipendentemente dal miglioramento delle metriche fisiche. Il *Numero medio di Iterazioni* quantifica invece quanti cicli di refactoring e feedback sono stati necessari agli agenti per convergere verso una soluzione valida, fornendo una misura della robustezza dell'orchestrazione e dell'efficacia del meccanismo di feedback del `ReviewerAgent`.